

- Description

commons-math-4.0-beta1-src\commons-math-
legacy\src\main\java\org\apache\commons\math4\legacy\analysis\solvers\MullerSolver
.java

- Specification

module MullerSolver

type

min, max, start: real // search interval [min, max] and optional start
maxEval: integer // maximum allowed function evaluations
relAcc, absAcc, fAcc: real // relative accuracy, absolute accuracy, function value
accuracy
r: real // returned root candidate
ex: set of {None, NullArgument, BadSequence, NoBracketing, TooManyEvaluations}

process solve(f, min, max, start, maxEval, relAcc, absAcc, fAcc: real) r: real, ex: set of {None, NullArgument, BadSequence, NoBracketing, TooManyEvaluations}

functional scenario form:

```
f = null && ex = NullArgument
||
min > max && ex = BadSequence
||
(¬bracketed) && ex = NoBracketing
||
yMin = 0 && ex = None && r = min
||
yMax = 0 && ex = None && r = max
||
yStart = 0 && ex = None && r = start
||
yStart * yMin < 0 && ex = None
    && r ∈ (min, start] && |f(r)| <= fAcc
||
yStart * yMax < 0 && ex = None
    && r ∈ [start, max) && |f(r)| <= fAcc
||
xNew - xPrev <= tol
    && ex = None
    && r = xNew && |f(r)| <= fAcc
||
xMid - xPrev <= tol
    && ex = None
    && r = xMid && |f(r)| <= fAcc
||
```

```
        evaluations > maxEval && ex = TooManyEvaluations
    end process
end module
```

- Java Code

```
/*
 * Licensed to the Apache Software Foundation (ASF) under one or more
 * contributor license agreements. See the NOTICE file distributed with
 * this work for additional information regarding copyright ownership.
 * The ASF licenses this file to You under the Apache License, Version
2.0
 * (the "License"); you may not use this file except in compliance with
 * the License. You may obtain a copy of the License at
 *
 *      http://www.apache.org/licenses/LICENSE-2.0
 *
 * Unless required by applicable law or agreed to in writing, software
 * distributed under the License is distributed on an "AS IS" BASIS,
 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or impl
ied.
 * See the License for the specific language governing permissions and
 * limitations under the License.
 */
package org.apache.commons.math4.legacy.analysis.solvers;
import org.apache.commons.math4.legacy.exception.NoBracketingException;
import org.apache.commons.math4.legacy.exception.NumberIsTooLargeExcept
ion;
import org.apache.commons.math4.legacy.exception.TooManyEvaluationsExce
ption;
import org.apache.commons.math4.core.jdkmath.JdkMath;
/**
 * This class implements the
 * <a href="http://mathworld.wolfram.com/MullersMethod.html"> Muller's
 * Method</a> for root finding of real univariate functions. For refere
nce, see
 * <b>Elementary Numerical Analysis</b>, ISBN 0070124477, chapter 3.
 * <p>
 * Muller's method applies to both real and complex functions, but here
we
 * restrict ourselves to real functions. This class differs from
 * {@link MullerSolver} in the way it avoids complex operations.
 * </p>
 * <p>
```

```

    * Muller's original method would have function evaluation at complex p
    oint.
    * Since our f(x) is real, we have to find ways to avoid that. Bracketi
    ng
    * condition is one way to go: by requiring bracketing in every iterati
    on, the
    * newly computed approximation is guaranteed to be real.
    * </p>
    * <p>
    * Normally Muller's method converges quadratically in the vicinity of
    a zero,
    * however it may be very slow in regions far away from zeros. For exam
    ple, f(x)
    * = exp(x) - 1, min = -
    50, max = 100. In such case we use bisection as a safety
    * backup if it performs very poorly.
    * </p>
    * <p>
    * The formulas here use divided differences directly.
    * </p>
    *
    * @since 1.2
    * @see MullerSolver2
    */
public class MullerSolver extends AbstractUnivariateSolver {
    /** Default absolute accuracy. */
    private static final double DEFAULT_ABSOLUTE_ACCURACY = 1e-6;
    /**
     * Construct a solver with default accuracy (1e-6).
     */
    public MullerSolver() {
        this(DEFAULT_ABSOLUTE_ACCURACY);
    }
    /**
     * Construct a solver.
     *
     * @param absoluteAccuracy Absolute accuracy.
     */
    public MullerSolver(double absoluteAccuracy) {
        super(absoluteAccuracy);
    }
    /**
     * Construct a solver.
     *

```

```

    * @param relativeAccuracy Relative accuracy.
    * @param absoluteAccuracy Absolute accuracy.
    */
    public MullerSolver(double relativeAccuracy, double absoluteAccuracy)
    {
        super(relativeAccuracy, absoluteAccuracy);
    }
    /**
     * {@inheritDoc}
     */
    @Override
    protected double doSolve() throws TooManyEvaluationsException, NumberI
sTooLargeException, NoBracketingException {
        final double min = getMin();
        final double max = getMax();
        final double initial = getStartValue();
        final double functionValueAccuracy = getFunctionValueAccuracy();
        verifySequence(min, initial, max);
        // check for zeros before verifying bracketing
        final double fMin = computeObjectiveValue(min);
        if (JdkMath.abs(fMin) < functionValueAccuracy) {
            return min;
        }
        final double fMax = computeObjectiveValue(max);
        if (JdkMath.abs(fMax) < functionValueAccuracy) {
            return max;
        }
        final double fInitial = computeObjectiveValue(initial);
        if (JdkMath.abs(fInitial) < functionValueAccuracy) {
            return initial;
        }
        verifyBracketing(min, max);
        if (isBracketing(min, initial)) {
            return solve(min, initial, fMin, fInitial);
        } else {
            return solve(initial, max, fInitial, fMax);
        }
    }
    /**
     * Find a real root in the given interval.
     *
     * @param min Lower bound for the interval.
     * @param max Upper bound for the interval.
     * @param fMin function value at the lower bound.

```

```

    * @param fMax function value at the upper bound.
    * @return the point at which the function value is zero.
    * @throws TooManyEvaluationsException if the allowed number of calls
to the
    *
function to be solved has been exhausted.
    */
private double solve(double min, double max, double fMin, double fMax)
throws TooManyEvaluationsException {
    final double relativeAccuracy = getRelativeAccuracy();
    final double absoluteAccuracy = getAbsoluteAccuracy();
    final double functionValueAccuracy = getFunctionValueAccuracy();
    // [x0, x2] is the bracketing interval in each iteration
    // x1 is the last approximation and an interpolation point in (x0, x2
)
    // x is the new root approximation and new x1 for next round
    // d01, d12, d012 are divided differences
    double x0 = min;
    double y0 = fMin;
    double x2 = max;
    double y2 = fMax;
    double x1 = 0.5 * (x0 + x2);
    double y1 = computeObjectiveValue(x1);
    double oldx = Double.POSITIVE_INFINITY;
    while (true) {
        // Muller's method employs quadratic interpolation through
        // x0, x1, x2 and x is the zero of the interpolating parabola.
        // Due to bracketing condition, this parabola must have two
        // real roots and we choose one in [x0, x2] to be x.
        final double d01 = (y1 - y0) / (x1 - x0);
        final double d12 = (y2 - y1) / (x2 - x1);
        final double d012 = (d12 - d01) / (x2 - x0);
        final double c1 = d01 + (x1 - x0) * d012;
        final double delta = c1 * c1 - 4 * y1 * d012;
        final double xplus = x1 + (-2.0 * y1) / (c1 + JdkMath.sqrt(delta));
        final double xminus = x1 + (-2.0 * y1) / (c1 - JdkMath.sqrt(delta));
        // xplus and xminus are two roots of parabola and at least
        // one of them should lie in (x0, x2)
        final double x = isSequence(x0, xplus, x2) ? xplus : xminus;
        final double y = computeObjectiveValue(x);
        // check for convergence
        final double tolerance = JdkMath.max(relativeAccuracy * JdkMath.abs(
x), absoluteAccuracy);

```

```

    if (JdkMath.abs(x - oldx) <= tolerance || JdkMath.abs(y) <= function
ValueAccuracy) {
        return x;
    }
    // Bisect if convergence is too slow. Bisection would waste
    // our calculation of x, hopefully it won't happen often.
    // the real number equality test x == x1 is intentional and
    // completes the proximity tests above it
    boolean bisect = (x < x1 && (x1 - x0) > 0.95 * (x2 - x0)) || (x > x1
&& (x2 - x1) > 0.95 * (x2 - x0))
        || (x == x1);
    // prepare the new bracketing interval for next iteration
    if (!bisect) {
        x0 = x < x1 ? x0 : x1;
        y0 = x < x1 ? y0 : y1;
        x2 = x > x1 ? x2 : x1;
        y2 = x > x1 ? y2 : y1;
        x1 = x;
        y1 = y;
        oldx = x;
    } else {
        double xm = 0.5 * (x0 + x2);
        double ym = computeObjectiveValue(xm);
        if (JdkMath.signum(y0) + JdkMath.signum(ym) == 0.0) {
            x2 = xm;
            y2 = ym;
        } else {
            x0 = xm;
            y0 = ym;
        }
        x1 = 0.5 * (x0 + x2);
        y1 = computeObjectiveValue(x1);
        oldx = Double.POSITIVE_INFINITY;
    }
}
}
}
}
// V-Method
class MSVT {
    private static boolean approx(double a, double b, double tol) {
        if (Double.isNaN(a) || Double.isNaN(b))
            return false;
        if (Double.isInfinite(a) || Double.isInfinite(b))
            return a == b;
    }
}

```

```

    return Math.abs(a - b) <= tol * Math.max(1.0, Math.max(Math.abs(a), Math.abs(b)));
}
private static void check(String name, boolean cond) {
    System.out.printf("[%s] %s\n", cond ? "PASS" : "FAIL", name);
}
public void Vtest() {
    System.out.println("=== MullerSolver V-method tests ===");
    MullerSolver solver = new MullerSolver(); // default tolerances
    // --- VT1: endpoint root at min:  $f(x)=x+2$  on  $[-2, 3] \rightarrow -2$ 
    try {
        org.apache.commons.math4.legacy.analysis.UnivariateFunction f = x ->
x + 2.0;
        double r = solver.solve(1000, f, -2.0, 3.0);
        System.out.printf("VT1 root=%.12f, f(root)=%.3e\n", r, f.value(r));
        check("VT1 == -2.0", approx(r, -2.0, 0));
    } catch (Throwable t) {
        t.printStackTrace();
        check("VT1 unexpected exception", false);
    }
    // --- VT2: endpoint root at max:  $f(x)=x-4$  on  $[2, 4] \rightarrow 4$ 
    try {
        org.apache.commons.math4.legacy.analysis.UnivariateFunction f = x ->
x - 4.0;
        double r = solver.solve(1000, f, 2.0, 4.0);
        System.out.printf("VT2 root=%.12f, f(root)=%.3e\n", r, f.value(r));
        check("VT2 == 4.0", approx(r, 4.0, 0));
    } catch (Throwable t) {
        t.printStackTrace();
        check("VT2 unexpected exception", false);
    }
    // --- VT3: start equals root (short-circuit on start)
    //  $f(x)=(x-1)(x-3)$ , start=1 in  $[0,5] \rightarrow 1$ 
    try {
        org.apache.commons.math4.legacy.analysis.UnivariateFunction f = x ->
(x - 1.0) * (x - 3.0);
        double r = solver.solve(1000, f, 0.0, 5.0, 1.0);
        System.out.printf("VT3 root=%.12f, f(root)=%.3e\n", r, f.value(r));
        check("VT3 == 1.0", approx(r, 1.0, 0));
    } catch (Throwable t) {
        t.printStackTrace();
        check("VT3 unexpected exception", false);
    }
    // --- VT4:  $yStart*yMin < 0 \rightarrow$  triage to (min, start]

```

```

// f(x)=(x-1)(x-3), min=0.5 (f>0), start=1.2 (f<0), max=1.3 → root ~1
try {
    org.apache.commons.math4.legacy.analysis.UnivariateFunction f = x ->
(x - 1.0) * (x - 3.0);
    double min = 0.5, start = 1.2, max = 1.3;
    double yMin = f.value(min), yStart = f.value(start);
    System.out.printf("VT4 signs: yMin=%.3f, yStart=%.3f\n", yMin, yStar
t);
    double r = solver.solve(1000, f, min, max, start);
    System.out.printf("VT4 root=%.12f, f(root)=%.3e\n", r, f.value(r));
    check("VT4 ~ 1.0", approx(r, 1.0, 1e-10));
} catch (Throwable t) {
    t.printStackTrace();
    check("VT4 unexpected exception", false);
}
// --- VT5: yStart*yMax < 0 → triage to [start, max)
// f(x)=(x-1)(x-3), min=1.5, start=2.8 (f<0), max=3.5 (f>0) → root ~3
try {
    org.apache.commons.math4.legacy.analysis.UnivariateFunction f = x ->
(x - 1.0) * (x - 3.0);
    double min = 1.5, start = 2.8, max = 3.5;
    double yStart = f.value(start), yMax = f.value(max);
    System.out.printf("VT5 signs: yStart=%.3f, yMax=%.3f\n", yStart, yMa
x);
    double r = solver.solve(1000, f, min, max, start);
    System.out.printf("VT5 root=%.12f, f(root)=%.3e\n", r, f.value(r));
    check("VT5 ~ 3.0", approx(r, 3.0, 1e-10));
} catch (Throwable t) {
    t.printStackTrace();
    check("VT5 unexpected exception", false);
}
// --- VT6: normal convergence inside bracket to mid root ~2
try {
    org.apache.commons.math4.legacy.analysis.UnivariateFunction f = x ->
(x - 1.0) * (x - 2.0) * (x - 3.0);
    double r = solver.solve(1000, f, 1.5, 2.5);
    System.out.printf("VT6 root=%.12f, f(root)=%.3e\n", r, f.value(r));
    check("VT6 ~ 2.0", approx(r, 2.0, 1e-10));
} catch (Throwable t) {
    t.printStackTrace();
    check("VT6 unexpected exception", false);
}
// --- VT7: No bracketing (should throw) – f(x)=x^2+1 on [-3,2] is >0
try {

```

```

    org.apache.commons.math4.legacy.analysis.UnivariateFunction f = x ->
x * x + 1.0;
    solver.solve(1000, f, -3.0, 2.0);
    check("VT7 should throw NoBracketing", false);
} catch (IllegalArgumentException e) {
    System.out.println("VT7 threw: " + e.getClass().getSimpleName());
    check("VT7 threw (not bracketed)", true);
} catch (Throwable t) {
    t.printStackTrace();
    check("VT7 wrong exception", false);
}
// --- VT8: Bad interval order (min > max) (should throw)
try {
    org.apache.commons.math4.legacy.analysis.UnivariateFunction f = x ->
x - 1.0;
    solver.solve(1000, f, 5.0, 1.0);
    check("VT8 should throw (bad sequence)", false);
} catch (IllegalArgumentException e) {
    System.out.println("VT8 threw: " + e.getClass().getSimpleName());
    check("VT8 threw (bad sequence)", true);
} catch (Throwable t) {
    t.printStackTrace();
    check("VT8 wrong exception", false);
}
// --- VT9: TooManyEvaluations (tiny maxEval) on a bracketed root
try {
    org.apache.commons.math4.legacy.analysis.UnivariateFunction f = x ->
Math.cos(x) - x; // root in [0,1]
    solver.solve(1, f, 0.0, 1.0, 0.5); // absurdly small budget
    check("VT9 should throw TooManyEvaluations", false);
} catch (IllegalStateException e) {
    System.out.println("VT9 threw: " + e.getClass().getSimpleName());
    check("VT9 threw (too many evals)", true);
} catch (Throwable t) {
    t.printStackTrace();
    check("VT9 wrong exception", false);
}
// --- VT10: Likely quadratic-
step rejection → fallback (e.g., bisection) then
// convergence
// Steep exponential can propose an out-of-
interval quadratic step; solver
// should fall back safely.
try {

```

```

    org.apache.commons.math4.legacy.analysis.UnivariateFunction f = x ->
Math.expm1(50.0 * x); // exp(50x)-1
    // Bracket a small root near x=0 with a skewed start.
    double r = solver.solve(200, f, -0.1, 0.1, 0.08);
    System.out.printf("VT10 root=%.12f, f(root)=%.3e%n", r, f.value(r));
    check("VT10 ~ 0.0", approx(r, 0.0, 1e-8));
} catch (Throwable t) {
    t.printStackTrace();
    check("VT10 unexpected exception", false);
}
// --- VT11:
try {
    org.apache.commons.math4.legacy.analysis.UnivariateFunction f = x ->
{
    double v = Math.exp(800.0 * x); // huge growth
    double rhs = 1.0e100; // very large offset
    return v - rhs;
};
    double r = solver.solve(200, f, -
1.0, 1.0, 0.0); // start at midpoint (NOT a root)
    System.out.printf("VT11 root=%.12f, f(root)=%.3e%n", r, f.value(r));
    // Expect a root very close to  $x \approx \ln(1e100)/800 \approx (\approx 230.2585)/800 \approx 0.288$ 
    check("VT11 ~ 0.29", approx(r, 0.288, 5e-2)); // Loose on purpose
} catch (Throwable t) {
    t.printStackTrace();
    check("VT11 unexpected exception", false);
}
// --- VT12:
try {
    org.apache.commons.math4.legacy.analysis.UnivariateFunction f = x ->
{
    double v = Math.exp(700.0 * x);
    double rhs = 1.0e70;
    return v - rhs;
};
    double r = solver.solve(300, f, -0.8, 0.8, 0.0);
    System.out.printf("VT12 root=%.12f, f(root)=%.3e%n", r, f.value(r));
    //  $\ln(1e70)/700 \approx (\approx 161.18096)/700 \approx 0.2303$ 
    check("VT12 ~ 0.23", approx(r, 0.230, 5e-2));
} catch (Throwable t) {
    t.printStackTrace();
    check("VT12 unexpected exception", false);
}

```

```

// --- VT13:
try {
    org.apache.commons.math4.legacy.analysis.UnivariateFunction f = x ->
{
    double v = Math.exp(1200.0 * x);
    return v - 1.0e200;
};
    double min = -1.0, max = 1.0, start = 0.0;
    double r = solver.solve(300, f, min, max, start);
    System.out.printf("VT13 root=%.12f, f(root)=%.3e%n", r, f.value(r));
    check("VT13 ~ 0.38 (x==x1 触发 bisection)", approx(r, 0.384, 6e-2));
} catch (Throwable t) {
    t.printStackTrace();
    check("VT13 unexpected exception", false);
}

// --- VT14:
try {
    org.apache.commons.math4.legacy.analysis.UnivariateFunction f = x ->
(x - 0.25) * (x - 0.9) * (x + 0.8);
    double min = -1.0, max = 1.0, start = 0.0;
    double r = solver.solve(300, f, min, max, start);
    System.out.printf("VT14 root=%.12f, f(root)=%.3e%n", r, f.value(r));
    boolean ok = approx(r, 0.25, 1e-8) || approx(r, -0.8, 1e-
8) || approx(r, 0.9, 1e-8);
    check("VT14 converged via Muller step (x!=x1)", ok);
} catch (Throwable t) {
    t.printStackTrace();
    check("VT14 unexpected exception", false);
}

// --- VT15:
try {
    org.apache.commons.math4.legacy.analysis.UnivariateFunction f = x ->
x - 1.0e-16;
    double r = solver.solve(200, f, -1.0e-3, 1.0e-
3, 0.0); // bracket: f(min)<0, f(max)>0, start=mid
    System.out.printf("VT15 root=%.12f, f(root)=%.3e%n", r, f.value(r));
    check("VT15 ~ 1e-16", approx(r, 1.0e-16, 1e-2));
} catch (Throwable t) {
    t.printStackTrace();
    check("VT15 unexpected exception", false);
}

// --- VT16:
try {

```

```

    org.apache.commons.math4.legacy.analysis.UnivariateFunction f = x ->
1e-20 * x * x + 1e-10 * x - 1e-20;
    double r = solver.solve(400, f, -1e-5, 1e-5, 0.0);
    System.out.printf("VT16 root=%.12f, f(root)=%.3e%n", r, f.value(r));
    check("VT16 small root", Math.abs(r) < 1e-4);
} catch (Throwable t) {
    t.printStackTrace();
    check("VT16 unexpected exception", false);
}
// --- VT17:
try {
    org.apache.commons.math4.legacy.analysis.UnivariateFunction f = x ->
Math.exp(1600.0 * x) - 1.0e250;
    double r = solver.solve(300, f, -1.0, 1.0, 0.0); // start=mid
    System.out.printf("VT17 root=%.12f, f(root)=%.3e%n", r, f.value(r));
    // ln(1e250)/1600 ≈ 575.646.../1600 ≈ 0.3598
    check("VT17 ~ 0.36", approx(r, 0.360, 6e-2));
} catch (Throwable t) {
    t.printStackTrace();
    check("VT17 unexpected exception", false);
}
// --- VT18:
try {
    org.apache.commons.math4.legacy.analysis.UnivariateFunction f = x ->
(x - 0.2) * (x + 0.7) * (x - 0.95);
    double r = solver.solve(300, f, -1.0, 1.0, -0.3);
    System.out.printf("VT18 root=%.12f, f(root)=%.3e%n", r, f.value(r));
    boolean ok = approx(r, 0.2, 1e-8) || approx(r, -0.7, 1e-
8) || approx(r, 0.95, 1e-8);
    check("VT18 Muller step (x!=x1)", ok);
} catch (Throwable t) {
    t.printStackTrace();
    check("VT18 unexpected exception", false);
}
// --- VT19:
try {
    org.apache.commons.math4.legacy.analysis.UnivariateFunction f = x ->
Math.exp(900.0 * x)
    - Math.exp(-900.0 * x);
    org.apache.commons.math4.legacy.analysis.UnivariateFunction g = x ->
f.value(x) + 1e-200;
    double r = solver.solve(400, g, -1.0, 1.0, 0.0);
    System.out.printf("VT19 root=%.12f, g(root)=%.3e%n", r, g.value(r));
    check("VT19 near 0", Math.abs(r) < 0.1);
}

```

```

    } catch (Throwable t) {
        t.printStackTrace();
        check("VT19 unexpected exception", false);
    }
    // --- VT20:
    try {
        org.apache.commons.math4.legacy.analysis.UnivariateFunction f = x ->
Math.tanh(20.0 * x) - 1e-6;
        double r = solver.solve(200, f, -0.5, 0.5, 0.1);
        System.out.printf("VT20 root=%.12f, f(root)=%.3e%n", r, f.value(r));
        check("VT20 ~ 0", Math.abs(r) < 0.1);
    } catch (Throwable t) {
        t.printStackTrace();
        check("VT20 unexpected exception", false);
    }
    System.out.println("=== Done MullerSolver V-method tests ===");
}
}
//===== MullerSolver FUZZ: 20 pure-random-
Looking cases =====
class MSFT {
    private static void runCase(String name, org.apache.commons.math4.legacy.analysis.UnivariateFunction f, double min,
        double max, int maxEval) {
        System.out.println("---- " + name + " (no start) ----");
        try {
            MullerSolver solver = new MullerSolver();
            double r = solver.solve(maxEval, f, min, max);
            System.out.printf("root=%.17g, f(root)=%.3e%n", r, f.value(r));
        } catch (Throwable t) {
            System.out.println("threw: " + t.getClass().getSimpleName() + " - "
+ t.getMessage());
        }
    }
    private static void runCaseWithStart(String name, org.apache.commons.math4.legacy.analysis.UnivariateFunction f,
        double min, double max, double start, int maxEval) {
        System.out.println("---- " + name + " (with start) ----");
        try {
            MullerSolver solver = new MullerSolver();
            double r = solver.solve(maxEval, f, min, max, start);
            System.out.printf("root=%.17g, f(root)=%.3e%n", r, f.value(r));
        } catch (Throwable t) {

```

```

        System.out.println("threw: " + t.getClass().getSimpleName() + " - "
+ t.getMessage());
    }
}

public void Ftest() {
    System.out.println("=== MullerSolver FUZZ: 20 cases ===");
    // FZ01: cubic-ish polynomial with jagged coefficients
    runCaseWithStart("FZ01",
        x -> -
0.773102211903 + 1.9922003117 * x - 0.003912200131 * x * x + 0.22319001
1 * x * x * x,
        -2.337110022, 3.119900177, 0.6679011001, 400);
    // FZ02: oscillatory + linear tilt
    runCase("FZ02", x -> Math.sin(4.66920160910299 * x) + 0.331700220019
* x - 0.0011772019, -6.129004728113,
        4.669201609102990, 600);
    // FZ03: steep exponential minus offset (can overflow near edges)
    runCaseWithStart("FZ03", x -> Math.exp(290.331901177 * x) - 3.8320231
39620673E126, -1.771, 1.771, -0.119001,
        250);
    // FZ04: hyperbolic tangent wiggle
    runCase("FZ04", x -> Math.tanh(57.9011772001 * x) - 0.000003311177, -
1.999, 1.999, 200);
    // FZ05: rational weirdness
    runCaseWithStart("FZ05", x -> x - 1.117720119 / (1.0 + 0.003901223117
* x + 2.2e-9 * x * x), -10.0, 10.0,
        -3.337200119, 500);
    // FZ06: trig-exponential cocktail
    runCase("FZ06", x -> Math.cos(19.7710022003 * x) - Math.exp(-
0.33170022 * x) + 0.091177201, -12.0, 12.0, 700);
    // FZ07: super-flat quadratic near origin
    runCaseWithStart("FZ07", x -> 1.0e-20 * x * x + 1.0e-10 * x - 3.0e-
22, -1.0e-4, 1.0e-4, 0.0, 800);
    // FZ08: asymmetric sigmoid
    runCase("FZ08", x -> 1.0 / (1.0 + Math.exp(-
23.1900117 * x)) - 0.500000331177, -2.0, 2.0, 350);
    // FZ09: sinh with offset (very steep)
    runCaseWithStart("FZ09", x -> Math.sinh(120.77119 * x) - 1.6309011038
425763E12, -0.5, 0.5, 0.0, 180);
    // FZ10: Chebyshev-like oscillation
    runCase("FZ10", x -> Math.cos(7.0 * Math.acos(Math.max(-
1.0, Math.min(1.0, x / 1.7)))) - 0.00331177, -1.7, 1.7,
        420);
    // FZ11: weird fractional power (careful at x<0)

```

```

    runCaseWithStart("FZ11", x -> Math.copySign(Math.pow(Math.abs(x + 0.3
3711), 1.3), x + 0.33711) - 0.119000177,
        -2.0, 2.0, -0.9, 300);
    // FZ12: cubic with extreme constant and tiny slope
    runCase("FZ12", x -> 3.832023139620673E294 + 1.1177e-
300 * x + x * x * x, -1.0, 1.0, 5);
    // FZ13: noisy cosine baseline
    runCaseWithStart("FZ13", x -> Math.cos(0.991177201 * x + 0.22310011)
+ 0.000000019 * x - 0.44, -50.0, 50.0,
        12.003119, 900);
    // FZ14: product of linear factors plus wavy term
    runCase("FZ14", x -> (x - 0.88300217) * (x + 0.771190011) * (x - 1.33
722001) + 0.11772011 * Math.sin(11.7 * x),
        -3.0, 3.0, 650);
    // FZ15: expm1 scaled (near-zero trickiness)
    runCaseWithStart("FZ15", x -> Math.expm1(60.002 * x) - 1.0e-7, -
0.3, 0.3, 0.02, 260);
    // FZ16: log(1+x) with shift (domain guarded)
    runCase("FZ16", x -> Math.log1p(Math.max(-
0.9999999999999999, x + 0.119001)) - 0.00390122, -0.5, 1.5, 400);
    // FZ17: tan with damping (can be nasty near poles)
    runCaseWithStart("FZ17", x -> Math.tan(0.77119 * x) - 0.01 * x, -
3.0, 3.0, 0.3, 550);
    // FZ18: damped oscillation
    runCase("FZ18", x -> Math.exp(-
0.3317002 * Math.abs(x)) * Math.sin(33.1177201 * x) - 0.00091, -
5.0, 5.0, 480);
    // FZ19: "almost linear" with tiny cubic correction
    runCaseWithStart("FZ19", x -> 1.000000003 * x + 1.0e-
12 * x * x * x - 0.000000119, -1.0, 1.0, -0.6, 320);
    // FZ20: heavy-tailed arctan blend
    runCase("FZ20", x -> Math.atan(9.91177201 * x) + 0.001 * x - 0.331700
22, -100.0, 100.0, 700);
    System.out.println("=== Done FUZZ ===");
}
}
//Z3
class MSZ3 {
    private static void runCase(String name, org.apache.commons.math4.lega
cy.analysis.UnivariateFunction f, double min,
        double max, int maxEval) {
        System.out.println("---- " + name + " (no start) ----");
        try {
            MullerSolver solver = new MullerSolver();

```

```

        double r = solver.solve(maxEval, f, min, max);
        System.out.printf("root=%.17g, f(root)=%.3e%n", r, f.value(r));
    } catch (Throwable t) {
        System.out.println("threw: " + t.getClass().getSimpleName() + " - "
+ t.getMessage());
    }
}

private static void runCaseWithStart(String name, org.apache.commons.m
ath4.legacy.analysis.UnivariateFunction f,
    double min, double max, double start, int maxEval) {
    System.out.println("---- " + name + " (with start) ----");
    try {
        MullerSolver solver = new MullerSolver();
        double r = solver.solve(maxEval, f, min, max, start);
        System.out.printf("root=%.17g, f(root)=%.3e%n", r, f.value(r));
    } catch (Throwable t) {
        System.out.println("threw: " + t.getClass().getSimpleName() + " - "
+ t.getMessage());
    }
}

// Build a linear function that hits f(min)=yMin and f(max)=yMax.
// If min==max, fallback to constant yMin.
private static org.apache.commons.math4.legacy.analysis.UnivariateFunc
tion linearThrough(double min, double yMin,
    double max, double yMax) {
    if (Double.compare(min, max) == 0) {
        final double c = yMin;
        return x -> c;
    }
    final double a = (yMax - yMin) / (max - min);
    final double b = yMin - a * min;
    return x -> a * x + b;
}

public void Z3test() {
    System.out.println("=== MullerSolver (Z3-derived) : 20 cases ===");
    // Row 1: f=null -> NullArgument (min=0, max=0)
    runCase("Z3-01 f=null -> NullArgument", null, 0.0, 0.0, 50);
    // Row 2: f=null -> NullArgument (min=-1, max=-1)
    runCase("Z3-02 f=null -> NullArgument", null, -1.0, -1.0, 50);
    // Row 3: min>max -> BadSequence (max=0, min=1/2)
    runCase("Z3-03 min>max -> BadSequence", x -> x - 1.0, 0.5, 0.0, 100);
    // Row 4: min>max -> BadSequence (max=-1/4, min=0)
    runCase("Z3-04 min>max -> BadSequence", x -> x + 0.25, 0.0, -
0.25, 100);
}

```

```

// Row 5: not bracketed -> NoBracketing (yMin=0 at min=-999, yMax=-
1 at max=0)
runCase("Z3-05 not bracketed -> NoBracketing", linearThrough(-
999.0, 0.0, 0.0, -1.0), -999.0, 0.0, 200);
// Row 6: not bracketed -> NoBracketing (yMin=-499 at min=-
999, yMax=0 at
// max=-999.5)
runCase("Z3-06 not bracketed -> NoBracketing", linearThrough(-
999.0, -499.0, -999.5, 0.0), -999.0, -999.5, 200);
// Row 7: yMin=0 -> r=min (min=-1, max=0)
runCase("Z3-07 yMin=0 -> r=min", x -> x - (-1.0), -1.0, 0.0, 200);
// Row 8: yMin=0 -> r=min (min=-2, max=0)
runCase("Z3-08 yMin=0 -> r=min", x -> x - (-2.0), -2.0, 0.0, 200);
// Row 9: yMax=0 -> r=max (min=0, max=1)
runCase("Z3-09 yMax=0 -> r=max", x -> x - 1.0, 0.0, 1.0, 200);
// Row 10: yMax=0 -> r=max (min=-999.5, max=-999)
runCase("Z3-10 yMax=0 -> r=max", x -> x - (-999.0), -999.5, -
999.0, 200);
// Row 11: start in interval & yStart=0 -> r=start (min=0,max=1,start
=0)
runCaseWithStart("Z3-
11 start in interval & yStart=0 -> r=start", x -> x - 0.0, 0.0, 1.0, 0.
0, 200);
// Row 12: start in interval & yStart=0 -> r=start (min=-
999.5,max=0,start=-999)
runCaseWithStart("Z3-
12 start in interval & yStart=0 -> r=start", x -> x - (-999.0), -
999.5, 0.0, -999.0, 200);
// Row 13: start*min opposite sign -> r∈(min,start] (min=-
999,start=0, choose
// max=1)
runCaseWithStart("Z3-13 start*min opposite sign", x -> x + 1.0, -
999.0, 1.0, 0.0, 200);
// Row 14: start*min opposite sign -> r∈(min,start] (min=-
999.5,start=-999,
// choose max=-998.5)
runCaseWithStart("Z3-14 start*min opposite sign", x -> x + 999.25, -
999.5, -998.5, -999.0, 200);
// Row 15: start*max opposite sign -> r∈[start,max) (max=1,start=0, c
hoose
// min=-1)
runCaseWithStart("Z3-15 start*max opposite sign", x -> x - 0.2, -
1.0, 1.0, 0.0, 200);

```

```

// Row 16: start*max opposite sign -> r∈[start,max) (max=-999,start=-
999.5,
// choose min=-1000)
runCaseWithStart("Z3-16 start*max opposite sign", x -> x + 999.25, -
1000.0, -999.0, -999.5, 200);
// Row 17: ∃xNew: in interval & (|f|≤fAcc or |Δ|≤tol) -> r=xNew
// (min=-1,max=0,start=0,xNew=0,xPrev=0) - simple root at 0
runCaseWithStart("Z3-17 exists xNew -> r=xNew", x -> x, -
1.0, 0.0, 0.0, 200);
// Row 18: ∃xNew: in interval & ... -> r=xNew
// (min=0,max=1,start=-1,xNew=1,xPrev=-1)
runCaseWithStart("Z3-
18 exists xNew -> r=xNew", x -> (x) * (x - 1.0), 0.0, 1.0, -1.0, 200);
// Row 19: quadraticStepInvalid -> pick xMid and r=xMid (min=-
1,max=0,xMid=-0.5)
// Use a very steep exponential to encourage Muller fallback behavior
.
runCase("Z3-
19 quadraticStepInvalid -> pick xMid", x -> Math.expm1(200.0 * x) - 1e-
20, -1.0, 0.0, 300);
// Row 20: evaluations>maxEval -> TooManyEvaluations
// (evaluations=1,maxEval=4,max=0,min=0)
// We still just run it as a concrete call with your bounds.
runCase("Z3-
20 evaluations>maxEval -> TooManyEvaluations", x -> x - 0.1, 0.0, 0.0,
4);
System.out.println("=== Done ===");
}
}
class MStest {
public static void main(String[] args) {
// VT
MSVT t1 = new MSVT();
t1.Vtest();
//      // FT
MSFT t2 = new MSFT();
//      t2.Ftest();
//      // Z3
MSZ3 t3 = new MSZ3();
//      t3.Z3test();
}
}

```

- Coverage report

1. Test case number: 20
2. V-Method testing branch coverage: 97.2%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ MullerSolver.java	71.4 %	50	20	70
> MSZ3	0.0 %	0	2	2
> MSVT	46.9 %	15	17	32
▼ MullerSolver	97.2 %	35	1	36
■ solve(double, double, double, double)	96.4 %	27	1	28
◆ doSolve()	100.0 %	8	0	8
● MullerSolver()		0	0	0
● MullerSolver(double)		0	0	0
> MSFT		0	0	0
> MStest		0	0	0

3. V-Method testing instruction coverage: 98.5%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ MullerSolver.java	49.7 %	1,177	1,193	2,370
> MSFT	0.6 %	3	493	496
> MSZ3	0.8 %	3	364	367
> MSVT	71.7 %	830	328	1,158
> MStest	83.3 %	15	3	18
▼ MullerSolver	98.5 %	326	5	331
● MullerSolver()	100.0 %	4	0	4
● MullerSolver(double)	100.0 %	4	0	4
◆ doSolve()	100.0 %	73	0	73
■ solve(double, double, double, double)	100.0 %	245	0	245

4. Fuzz testing branch coverage: 30.6%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ MullerSolver.java	17.1 %	12	58	70
> MSVT	0.0 %	0	32	32
▼ MullerSolver	30.6 %	11	25	36
■ solve(double, double, double, double)	10.7 %	3	25	28
◆ doSolve()	100.0 %	8	0	8
● MullerSolver()		0	0	0
● MullerSolver(double)		0	0	0
> MSZ3	50.0 %	1	1	2
> MSFT		0	0	0
> MStest		0	0	0

5. Fuzz testing instruction coverage: 62.8%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ MullerSolver.java	23.8 %	565	1,805	2,370
> MSVT	0.3 %	3	1,155	1,158
> MSFT	0.6 %	3	493	496
▼ MullerSolver	62.8 %	208	123	331
■ solve(double, double, double, double)	51.8 %	127	118	245
● MullerSolver()	100.0 %	4	0	4
● MullerSolver(double)	100.0 %	4	0	4
◆ doSolve()	100.0 %	73	0	73
> MStest	83.3 %	15	3	18
> MSZ3	91.6 %	336	31	367

6. Z3 branch coverage: 91.7%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ MullerSolver.java	47.1 %	33	37	70
> MSVT	0.0 %	0	32	32
> MSZ3	0.0 %	0	2	2
▼ MullerSolver	91.7 %	33	3	36
◆ doSolve()	75.0 %	6	2	8
■ solve(double, double, double, double)	96.4 %	27	1	28
● MullerSolver()		0	0	0
● MullerSolver(double)		0	0	0
> MSFT		0	0	0
> MStest		0	0	0

7. Z3 instruction coverage: 97.3%

Element		Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼  MullerSolver.java		 35.4 %	839	1,531	2,370
>  MSVT		 0.3 %	3	1,155	1,158
>  MSZ3		 0.8 %	3	364	367
>  MStest		83.3 %	15	3	18
▼  MullerSolver		 97.3 %	322	9	331
 doSolve()		 94.5 %	69	4	73
 MullerSolver()		100.0 %	4	0	4
 MullerSolver(double)		100.0 %	4	0	4
 solve(double, double, double, d		 100.0 %	245	0	245
>  MSFT		 100.0 %	496	0	496